

Python Notes

Fundamentals to the Standard Library

A complete reference for the Python training sessions.

Read a section, then run the examples yourself — Python is learned at the keyboard.

Habilelabs · Python Training · September 2026

Contents

1. Getting Started — Installing & Running Python
2. How Python Works Under the Hood
3. Basics — Variables, Types & Strings
4. Truthiness & Operators
5. Control Flow
6. Data Structures — Lists, Tuples, Dicts & Sets
7. Comprehensions & Slicing
8. Functions
9. Functional Tools — Iterators & Generators
10. Object-Oriented Programming
11. Errors, Exceptions & Context Managers
12. Working with JSON
13. Files & Paths
14. Modules & Imports
15. Type Hints
16. Async Python (asyncio)
17. Standard Library Tour

1. Getting Started — Installing & Running Python

Before you write any Python, you need Python on your machine and a few everyday commands: how to run a script, how to install packages, and how to keep each project's packages separate from the others. That is this whole section — set it up once, and you will not think about it again.

Installing Python

- **Windows:** download the installer from `python.org/downloads` . On the first screen **tick "Add python.exe to PATH"** — almost every "python is not recognized" error comes from skipping this.
- **macOS:** macOS ships with Python 3 (`python3`). For a newer version use the python.org installer or Homebrew: `brew install python` .
- **Linux (Ubuntu/Debian):** `sudo apt install python3 python3-pip python3-venv` .

Verify the install from a terminal:

```
python3 --version      # e.g. Python 3.9.6    (on Windows: python --version)
pip3 --version         # the package installer that came with it
```

Running Python code

```
python3                # interactive REPL – type code, see results, exit()
python3 script.py      # run a file
python3 -i script.py   # run a file, then drop into the REPL with its variables
python3 -m http.server # run a stdlib module as a program (-m = "as module")
```

pip — installing packages

```
pip install requests      # install one package
pip install -r requirements.txt # install everything a project lists
pip list                  # what's installed
pip freeze > requirements.txt # snapshot current versions
```

Virtual environments — one per project

A virtual environment (venv) is a private, per-project copy of Python and its packages. Without one, every project shares the same global packages and version conflicts are guaranteed. **Always work inside a venv.**

```
python3 -m venv venv      # create it (a folder literally named venv/)
source venv/bin/activate  # activate – macOS / Linux
venv\Scripts\activate     # activate – Windows
(venv) $ pip install ...  # installs now go INTO the venv only
deactivate                # leave it
```

Tip: the `(venv)` prefix in your prompt tells you it is active. Never commit the `venv/` folder to git — commit `requirements.txt` instead, so anyone can rebuild it.

2. How Python Works Under the Hood

You can write Python without knowing any of this — but knowing it turns many "strange" behaviours into obvious ones. Why does order matter? Why does changing one list change "another" one too? Why does a file named `random.py` break everything? This section explains what actually happens between saving your file and seeing its output.

From your file to running code

When you run `python3 script.py`, the standard interpreter (called **CPython**, because it is written in C) does this:

1. **Tokenize & parse** — the source text is read and turned into a syntax tree. Syntax errors are caught here, before anything runs.
2. **Compile to bytecode** — the tree is compiled into *bytecode*: compact, low-level instructions for Python's own virtual machine (not for your CPU).
3. **Execute** — the **Python Virtual Machine (PVM)** runs the bytecode instruction by instruction, top to bottom.

So Python is *compiled* (to bytecode) *and* interpreted (the PVM executes that bytecode). Imported modules cache their bytecode in a `__pycache__` folder as `.pyc` files so the compile step is skipped next time. You can see the bytecode yourself:

```
import dis

def add(a, b):
    return a + b

dis.dis(add)
#  LOAD_FAST  a
#  LOAD_FAST  b
#  BINARY_ADD
#  RETURN_VALUE
```

Execution is top-to-bottom, statement by statement

There is no separate "declaration phase". The module runs from the first line to the last, and **everything is a statement that executes** — including `def` and `class`:

```
print("this runs first")

def greet():          # executing THIS line creates a function object
    print("hello")    # (the body does NOT run yet)

greet()               # only now does the body run

# Calling a function before its def line is a NameError –
# the name simply doesn't exist yet. Order matters.
```

An `import` statement runs the imported file's top-level code — once. The result is cached in `sys.modules`, so importing the same module again is free.

Names and objects — variables are labels

In Python, **everything is an object** — numbers, strings, functions, classes, modules. A variable does not *contain* a value; it is a *name bound to* an object. Assignment attaches a label, it never copies:

```
a = [1, 2, 3]
b = a          # b is a second label on the SAME list object
b.append(4)
print(a)       # [1, 2, 3, 4]
print(id(a) == id(b)) # True – same object identity
```

Gotcha: this is why copying matters for mutable objects (lists, dicts, sets). `b = a.copy()` makes a new object; `b = a` never does.

Dynamic but strongly typed

- **Dynamic:** types are checked while the program runs, not before. A name can be rebound to a different type at any time.
- **Strong:** Python refuses to silently mix types — `"1" + 1` raises `TypeError` instead of guessing.

Memory management & the GIL (short version)

- Objects are freed automatically by **reference counting** (when the last name pointing at an object goes away, the object is freed), plus a cycle-detecting garbage collector. You never `free()` anything.
- The **GIL** (Global Interpreter Lock) lets only one thread execute Python bytecode at a time. Threads are still great for *waiting* (network, disk); for CPU-heavy work use `multiprocessing`, and for many concurrent I/O tasks use `asyncio` (section 16).

Indentation IS the syntax

Python has no braces — the indentation defines the block structure, and mixing widths is a `IndentationError`. Use **4 spaces**, never tabs, and let your editor enforce it.

3. Basics — Variables, Types & Strings

Every program starts with data: numbers, text, yes/no flags, "nothing yet". A **variable** is simply a name you give to a piece of data so you can use it later. Python does not ask you to declare a type — you just assign, and Python figures out the type from the value. This section covers the built-in types you will touch every single day, and the string tools you will use most.

Variables

```
name = "Priya"      # no let/var/const — just assign
age = 22
a, b = 1, 2         # multiple assignment
a, b = b, a         # swap — right side is fully evaluated first
first, *rest = [1, 2, 3, 4] # unpacking: first=1, rest=[2, 3, 4]
```

Naming convention: `snake_case` for variables and functions, `PascalCase` for classes, `UPPER_SNAKE` for constants (constants are convention only — nothing stops reassignment).

The core built-in types

Type	Example	Notes
<code>int</code>	<code>42</code>	arbitrary precision — never overflows
<code>float</code>	<code>3.14</code>	usual floating point ($0.1 + 0.2 \neq 0.3$ here too)
<code>str</code>	<code>"hi"</code>	immutable text; no separate char type
<code>bool</code>	<code>True / False</code>	capitalised! a subclass of int
<code>NoneType</code>	<code>None</code>	the single "no value" — test with <code>is None</code>
<code>list / tuple / dict / set</code>	<code>[1] (1,) {"a":1} {1,2}</code>	section 6

Division — the classic surprise

```
7 / 2    # 3.5    true division, ALWAYS a float
7 // 2   # 3      floor division (rounds down: -7 // 2 == -4)
7 % 2    # 1      remainder
2 ** 10  # 1024   power
```

Casting

```
int("42")      # 42          int("4.2") raises ValueError — go via float()
float("4.2")    # 4.2
str(42)        # "42"
int("abc")     # ValueError — always validate user input
```

Strings

```
s = "hello world"
s[0]      # 'h'      indexing
s[-1]     # 'd'      negative = from the end
s[0:5]    # 'hello'  slicing (end exclusive)
s[::-1]   # 'dlrow olleh' reversed
len(s)    # 11
```

Strings are **immutable** — every method returns a *new* string:

```
s.upper() / s.lower() / s.title()
s.strip()                # trim whitespace (lstrip / rstrip too)
s.replace("world", "there")
s.split()                # ['hello', 'world'] (split(",") etc.)
" ".join(["a", "b"])     # 'a b' — join belongs to the SEPARATOR
s.startswith("he") / s.endswith("ld")
s.find("wor")            # 6, or -1 if missing ("wor" in s is nicer)
"42".isdigit()           # True
```

f-strings — use them everywhere

```
name, price = "Chai", 15.5
f"{name} costs {price} rupees"    # interpolation
f"{price:.2f}"                    # '15.50'      2 decimals
f"{1234567:,}"                    # '1,234,567' thousands separator
f"{0.0876:.2%}"                   # '8.76%'     percentage
f"{name:>10}"                      # '      Chai' right-align to width 10
f"{name!r}"                       # "'Chai'"     repr — great for debugging
f"{price=}"                       # 'price=15.5' name AND value (3.8+)
```

Input & output

```
print("a", "b", sep="-", end="!\n") # a-b!
answer = input("How many? ")        # ALWAYS returns a string
count = int(answer)                  # cast it yourself
```

The entry-point guard

```
if __name__ == "__main__":
    main()
# __name__ is "__main__" only when this file is run directly.
# When the file is imported, __name__ is the module's name, so main()
# doesn't fire. Put run-only code behind this guard in every script.
```

4. Truthiness & Operators

Half of real code is making small decisions: is this list empty? did the user type anything? does this key exist? Python lets *any* value stand in for true or false inside a condition — that is called **truthiness**. Learn the short list of "falsy" values once, and your if-statements become short, readable, and safe.

Boolean operators are words

```
x and y      # not &&
x or y       # not ||
not x        # not !
```

What is falsy?

Exactly these: `False`, `None`, `0`, `0.0`, `""`, `[]`, `()`, `{}`, `set()`. **Everything else is truthy** — including `"false"`, `"0"`, and `[0]`.

```
items = []
if not items:           # idiomatic "is it empty?"
    print("empty")
```

`==` VS `is`

- `==` compares **values** (can be customised by classes).
- `is` compares **identity** — the same object in memory.
- Rule: use `is` *only* for `None`: `if x is None:` / `if x is not None:`.

Chained comparisons

```
if 0 <= score <= 100:    # reads like maths – both conditions checked
    ...
```

`in` — membership

```
"py" in "python"         # True (substring)
3 in [1, 2, 3]           # True
"name" in {"name": "P"}  # True – checks KEYS
x not in collection      # the negative form
```

Short-circuiting and the default trap

```
name = user_input or "guest"  # fallback if user_input is falsy
```

Gotcha: `or` replaces *every* falsy value — so a legitimate `0` or `""` also gets replaced. When only `None` should trigger the default, be explicit: `value if value is not None else default`.

Walrus operator `:=` (3.8+)

```
while (line := input("> ")) != "quit":    # assign AND test in one expression
    print(line.upper())
```

5. Control Flow

Control flow decides *which* lines run, in *what order*, and *how many times*. Python keeps the toolbox small: `if / elif / else` for decisions, `for` and `while` for repetition. The skill to build here is choosing the cleanest tool — a dict instead of a long `elif`-chain, `enumerate` instead of a manual counter, an early return instead of deep nesting.

if / elif / else

```
if score >= 90:
    grade = "A"
elif score >= 75:
    grade = "B"
else:
    grade = "C"
```

Ternary (conditional expression)

```
label = "item" if count == 1 else "items"    # value-if-true FIRST
```

No switch — use a dict

```
OPS = {"add": lambda a, b: a + b,
       "sub": lambda a, b: a - b}
fn = OPS.get(op)
result = fn(a, b) if fn else f"unknown op {op!r}"
```

(Python 3.10+ has `match/case`, but on 3.9 the dict-dispatch pattern is the tool.)

Loops

```
for i in range(5):          # 0..4          range(2, 10, 2) -> 2,4,6,8
    ...

for item in items:          # iterate ANY collection directly – no index needed
    ...

for i, item in enumerate(items, start=1):  # need the index? enumerate
    print(f"{i}. {item}")

for k, v in prices.items():  # dicts: unpack pairs
    ...

for name, score in zip(names, scores):    # two lists in lockstep
    ...

while attempts < 3:
    ...
```

`break` exits the loop, `continue` skips to the next iteration.

for / else — a Python special

```
for u in users:
    if u["id"] == target:
        print("found")
        break
else:
    # runs ONLY if the loop finished WITHOUT break
    print("not found")
```

Loop pitfalls

Gotcha: never add/remove items of a list *while* looping over it — iterate over a copy (`for x in items[:]`) or build a new list with a comprehension instead.

6. Data Structures — Lists, Tuples, Dicts & Sets

Almost every program is "take some data, organise it, transform it". Python gives you four built-in containers, and each has one clear job. Choosing the right one is most of the battle:

Container	Use it when you need...	Everyday example
<code>list</code>	an ordered collection that can grow and change	items in a shopping cart
<code>tuple</code>	a small fixed group of values that belong together	a coordinate (x, y); returning two values
<code>dict</code>	to look values up by a key, instantly	a user profile: name → value
<code>set</code>	unique values and fast "have I seen this?" checks	tags, visited IDs

list — ordered, mutable

```
nums = [3, 1, 4]
nums.append(5)           # add at end
nums.insert(0, 9)        # add at index
nums.extend([6, 7])      # add many
nums.remove(4)           # remove first matching VALUE (ValueError if absent)
last = nums.pop()        # remove & return last (pop(0) = first)
nums.index(1)            # position of value
nums.count(1)            # occurrences
nums.sort()              # in place; sorted(nums) returns a NEW list
nums.reverse()           # in place; nums[::-1] returns a new one
sum(nums), min(nums), max(nums), len(nums)
```

The copy gotcha: `b = a` aliases the same list. Independent copy: `b = a.copy()` or `b = a[:]`. These are *shallow* — nested lists are still shared (`copy.deepcopy` if needed).

tuple — ordered, immutable

```
point = (3, 5)
x, y = point              # unpacking
single = (42,)           # one item NEEDS the comma

def min_max(nums):        # THE tuple use-case:
    return min(nums), max(nums)  # return several values
low, high = min_max([4, 1, 9])
```

Tuples are **hashable**, so they can be dict keys and set members — lists cannot (`TypeError: unhashable type`).

```
cache = {(user_id, endpoint): response}  # composite keys — very useful
```

dict — key/value, insertion-ordered

```
user = {"name": "Priya", "age": 22}
user["name"]          # read – KeyError if missing
user.get("email")     # None if missing (no crash)
user.get("email", "n/a") # with a default
user["city"] = "Jaipur" # add / overwrite
del user["age"]        # remove
user.pop("age", None)  # remove with default – no crash

for k in user: ...      # keys
for v in user.values(): ...
for k, v in user.items(): ... # pairs – most common

merged = {**defaults, **overrides} # later dicts win
{k: v * 2 for k, v in prices.items()} # dict comprehension
```

Safe access into nested (JSON-shaped) data:

```
city = data.get("user", {}).get("address", {}).get("city") # None if any level missing
```

set — unique, unordered, fast membership

```
tags = {"python", "api"}
tags.add("json")
tags.discard("api") # no error if absent (remove() raises)

a, b = {1, 2, 3}, {2, 3, 4}
a & b    # intersection {2, 3}
a | b    # union        {1, 2, 3, 4}
a - b    # difference    {1}
a ^ b    # either-not-both {1, 4}

x in big_set    # effectively O(1) – convert a list you search often
```

Gotcha: `{}` is an empty **dict**. An empty set is `set()`.

collections: defaultdict & Counter

```
from collections import defaultdict, Counter

groups = defaultdict(list) # missing key -> fresh list, automatically
for p in people:
    groups[p["dept"]].append(p["name"])

freq = Counter("habilelabs") # counts anything iterable
freq.most_common(2)          # [('a', 2), ('b', 2)]
```

7. Comprehensions & Slicing

Two signature Python moves. A **comprehension** builds a new collection from an existing one in a single readable line — it replaces the common "make an empty list, loop, append" pattern. **Slicing** cuts a piece out of any sequence (list, string, tuple) by position. Together they remove most of the small loops you would otherwise write.

Comprehensions — build collections in one expression

```
squares = [n * n for n in nums]           # map
evens   = [n for n in nums if n % 2 == 0] # filter
labels  = [n if n > 0 else 0 for n in nums] # transform with ternary
flat     = [x for row in matrix for x in row] # nested (outer first)
by_name = {u["name"]: u["age"] for u in users} # dict comprehension
unique  = {e.lower() for e in emails}       # set comprehension
lazy    = (n * n for n in nums)             # GENERATOR expr – no list built
```

Rule of thumb: one condition + one transform = perfect comprehension. If it needs multiple lines to be readable, write a plain loop.

Slicing — `seq[start:stop:step]`

Read it as: *start here, stop before there, jumping this many at a time*. Leaving a part empty means "from the beginning", "to the end", or "step 1". A negative index counts from the end; a negative step walks backwards.

```
nums = [0, 1, 2, 3, 4, 5]
nums[1:4]      # [1, 2, 3]      stop is exclusive
nums[:3]       # [0, 1, 2]      from the start
nums[3:]       # [3, 4, 5]      to the end
nums[-2:]      # [4, 5]        last two
nums[::2]      # [0, 2, 4]      every 2nd
nums[::-1]     # reversed copy
nums[10:20]    # []             out of range is SAFE, never an error

nums[2:] + nums[:2]      # rotate left by 2
text.split()[::-1]       # reverse word order (then " ".join(...))
```

8. Functions

A function is a named, reusable block of logic: give it inputs, get an output. It is the main building block you will write all day — small functions that each do one thing, combined into bigger ones. Python functions are easy to define but come with powerful extras (keyword arguments, flexible signatures, functions as values) and one famous trap covered below.

Defining & calling

```
def greet(name, greeting="Hello"):    # default parameter
    """One-line docstring saying what it does."""
    return f"{greeting}, {name}!"

greet("Priya")                       # positional
greet(name="Priya", greeting="Hi")   # keyword – order doesn't matter
greet(greeting="Hi", name="Priya")   # same call
```

No return statement (or bare `return`) means the function returns `None`.

Keyword-only parameters

```
def format_currency(amount, *, symbol="₹", decimals=2):
    return f"{symbol}{amount:,.{decimals}f}"

format_currency(100, "₹")             # TypeError – everything after * is keyword-only
format_currency(100, symbol="$")     # OK – forces readable call sites
```

The mutable default trap

```
def add_item(item, bucket=[]):        # BUG: default created ONCE, at def time
    bucket.append(item)
    return bucket
add_item("a")    # ['a']
add_item("b")    # ['a', 'b'] ← the same list survived between calls!

def add_item(item, bucket=None):      # the fix – the None-default pattern
    if bucket is None:
        bucket = []
    ...
```

*args and **kwargs

```
def describe(*args, **kwargs):
    # args    = tuple of extra positional arguments
    # kwargs = dict of extra keyword arguments
    print(len(args), "positional,", len(kwargs), "keyword")

describe(1, 2, x=3)          # 2 positional, 1 keyword

# The reverse – SPREADING into a call:
nums = [3, 5]
add(*nums)                   # add(3, 5)
opts = {"symbol": "$"}
format_currency(9, **opts)   # format_currency(9, symbol="$")
```

Functions are values

```
ops = {"double": lambda n: n * 2}    # store them
def apply(fn, value):                # pass them
    return fn(value)
apply(str.upper, "hi")               # 'HI'
```

Lambdas — one expression only

```
sorted(users, key=lambda u: u["age"])    # their natural home: key=
sorted(users, key=lambda u: (-u["age"], u["name"])) # multi-key: a tuple
```

Closures & the late-binding trap

A **closure** is an inner function that remembers the variables of the function that created it — even after that outer function has finished. It is how a function can carry private state around without a class. The trap: it remembers the *variable*, not the value the variable had at the time.

```
def counter():
    count = 0
    def inc():
        nonlocal count    # write access to the enclosing variable
        count += 1
        return count
    return inc

fns = [lambda: i for i in range(3)]
[f() for f in fns]         # [2, 2, 2] – closures capture the VARIABLE, not the value
fns = [lambda i=i: i for i in range(3)] # fix: default freezes the value NOW
```

Decorators (preview)

A **decorator** wraps a function with extra behaviour — logging, timing, caching, retrying — without editing the function itself. The `@name` line above a `def` is just shorthand for "pass this function through that wrapper". You will *use* decorators constantly (`@property` , `@dataclass` , `@lru_cache`) long before you need to write your own.

```
import functools

def log_calls(fn):
    @functools.wraps(fn)                # keep fn's name/docstring
    def wrapper(*args, **kwargs):
        print(f"calling {fn.__name__}")
        return fn(*args, **kwargs)
    return wrapper

@log_calls                             # greet = log_calls(greet)
def greet(name):
    return f"hi {name}"
```

9. Functional Tools — Iterators & Generators

This section is about working on *whole collections* at once — transforming, filtering, sorting, aggregating — and about **laziness**: producing values only at the moment they are needed. The key idea is the **iterator**: think of it as a ticket dispenser. It hands out one value at a time, it cannot rewind, and when it is empty, it is empty. Laziness is what lets Python process files and datasets far bigger than your RAM.

map / filter are LAZY

```
m = map(str.upper, ["a", "b"]) # nothing computed yet – m is an iterator
list(m) # ['A', 'B'] – consumed now
list(m) # [] – an iterator is single-use!
```

In practice, comprehensions usually read better than `map / filter`:

```
[u["name"] for u in users if u["active"]] # filter + map in one pass
```

reduce — and the built-ins that replace it

```
from functools import reduce
total = reduce(lambda acc, n: acc + n, nums, 0) # works, but...
total = sum(nums) # ...prefer the built-in

sum(nums) / len(nums) # average
any(n > 10 for n in nums) # like "does any match?"
all(n > 0 for n in nums) # like "do all match?"
max(users, key=lambda u: u["age"]) # richest single pick
```

sorted — the full picture

```
sorted(nums) # new list, ascending
sorted(nums, reverse=True)
sorted(words, key=str.lower) # case-insensitive
sorted(users, key=lambda u: (-u["age"], u["name"])) # age DESC, name ASC
# .sort() does the same in place and returns None. Sorting is STABLE:
# equal keys keep their original relative order.
```

Generators — produce values on demand

```
def every_nth(iterable, n):
    for i, item in enumerate(iterable):
        if i % n == 0:
            yield item # pause here; resume on the next request

from itertools import islice
list(islice(every_nth(range(10**12), 5), 3)) # [0, 5, 10] – instant!
```

`yield` makes a function a generator: calling it builds a lazy iterator, and nothing executes until values are requested. This is how you process huge or infinite sequences in constant memory.

itertools & heapq greatest hits

```
from itertools import chain, product, islice
chain([1, 2], [3])          # one stream from many iterables
product(sizes, colours)     # all combinations
islice(gen, 5)              # first 5 of any iterator

import heapq
heapq.nlargest(2, users, key=lambda u: u["salary"])  # top-k WITHOUT full sort
```

10. Object-Oriented Programming

Object-oriented programming (OOP) groups data and the functions that work on that data into one unit: a **class**. The class is the blueprint (`User` — every user has a name and an email, and can greet); each **object** made from it is one concrete thing (the user Priya). In day-to-day work you will mostly *use* classes that libraries provide — but you must be able to read and write them comfortably, and to recognise the handful of patterns below.

Classes — the basic shape

```
class User:
    """A registered user."""

    def __init__(self, name, email):    # constructor
        self.name = name               # instance attributes
        self.email = email

    def greet(self):                   # self = this instance, always explicit
        return f"Hi, I'm {self.name}"

u = User("Priya", "p@x.io")           # no `new` keyword
u.greet()
```

Instance vs class attributes — the shared-list bug

```
class Team:
    members = []                      # BUG: ONE list shared by every instance!
    def add(self, name):
        self.members.append(name)

class Team:                           # FIX: create per-instance state in __init__
    def __init__(self):
        self.members = []
```

Class attributes are fine for constants; mutable state belongs in `__init__`.

Privacy is a convention

```
self._balance    # single underscore: "internal – please don't touch"
self.__secret    # double underscore: name-mangled (harder, not impossible)
```

@property, @classmethod, @staticmethod

Three kinds of methods, three jobs: a **property** is a computed value that *reads like an attribute* (no parentheses); a **classmethod** receives the class instead of an instance — its classic use is an alternative constructor ("build a `User` from a CSV line"); a **staticmethod** is a plain helper that just happens to live inside the class for organisation.

```

class User:
    def __init__(self, first, last):
        self.first, self.last = first, last

    @property
    # computed attribute – no parentheses to call
    def full_name(self):
        return f"{self.first} {self.last}"

    @classmethod
    # alternative constructor – gets the CLASS
    def from_csv(cls, line):
        first, last = line.split(",")
        return cls(first, last)

    @staticmethod
    # utility – gets neither instance nor class
    def is_valid_email(s):
        return "@" in s

User("Priya", "S").full_name    # 'Priya S'
User.from_csv("Priya,S")

```

Inheritance

```

class Animal:
    def __init__(self, name):
        self.name = name
    def speak(self):
        return "..."

class Dog(Animal):
    def __init__(self, name, breed):
        super().__init__(name)    # call the parent's constructor
        self.breed = breed
    def speak(self):
        # override
        return "woof"

isinstance(dog, Animal)    # True – respects inheritance; use over type(x) ==

```

Abstract base classes & duck typing

```

from abc import ABC, abstractmethod

class Shape(ABC):
    @abstractmethod
    def area(self): ...           # subclasses MUST implement

Shape()                          # TypeError – can't instantiate an ABC

```

Duck typing: most Python code doesn't check types at all — it just calls the method. Any object with a `speak()` works in a function that calls `thing.speak()`, no shared base class required. "If it quacks, it's a duck."

Design tip: prefer *composition* ("a Car HAS an Engine") over deep inheritance trees ("a Car IS a Vehicle IS a Machine ...").

Dataclasses — kill the boilerplate

```
from dataclasses import dataclass, field

@dataclass
class Product:
    name: str
    price: float
    tags: list = field(default_factory=list)    # NEVER tags: list = []

# You get __init__, a readable __repr__, and value-based __eq__ for free.

@dataclass(frozen=True)    # immutable -> also hashable (dict keys, set members)
class Money:
    amount: float
    currency: str = "INR"
```

Dunder methods — behave like a built-in

"Dunder" = *double underscore*. These specially-named methods are how your class plugs into Python's own syntax: define `__len__` and `len(your_object)` works; define `__add__` and `+` works. Python translates the nice syntax into these method calls:

You write	Python calls
<code>len(cart)</code>	<code>cart.__len__()</code>
<code>"apple" in cart</code>	<code>cart.__contains__("apple")</code>
<code>a + b</code>	<code>a.__add__(b)</code>
<code>a == b</code>	<code>a.__eq__(b)</code>
<code>print(x)</code>	<code>x.__str__()</code> (falls back to <code>__repr__</code>)
<code>for item in x</code>	<code>x.__iter__()</code>

11. Errors, Exceptions & Context Managers

Things go wrong: files are missing, user input is rubbish, networks time out. An **exception** is Python's signal that an operation failed — it stops normal execution and travels up through the callers until something catches it (or the program exits with a traceback). Good code *expects* the failures it can do something about, handles exactly those, and lets everything else crash loudly with a clear message — a hidden error is far worse than a visible one.

The full try shape

```
try:
    value = int(raw)
except ValueError as e:      # runs only on that error
    print(f"bad input: {e}")
except (KeyError, IndexError): # several types at once
    print("lookup failed")
else:                       # runs only if NO exception happened
    print("parsed fine")
finally:                    # runs ALWAYS – even after return or a re-raise
    print("cleanup")
```

Gotcha: `finally` runs even when the `try` block returns — the return value is computed first, then `finally` executes, then the function actually returns.

Raising

```
raise ValueError("weight cannot be negative")

try:
    parse(line)
except ValueError as e:
    raise ParseError(f"bad line {line!r}") from e  # keep the original cause!
```

Custom exceptions

```
class AppError(Exception):
    """Base for this app's errors."""

class ParseError(AppError):
    pass

# callers can catch ParseError specifically, or AppError for everything
```

The hierarchy worth remembering

```
Exception
├── ValueError      bad value      int("abc")
├── TypeError       bad type       "1" + 1
├── KeyError        missing dict key
├── IndexError      list index out of range
├── AttributeError  obj.missing_attr
├── FileNotFoundError open() a path that doesn't exist
└── ZeroDivisionError 1 / 0
```

- Catch the **narrowest** type you can actually handle.
- Never `except Exception: pass` — a silenced error is a hidden bug.

EAFP over LBYL

Two styles of handling "this might not work". **LBYL** ("look before you leap") checks first, then acts. **EAFP** ("easier to ask forgiveness than permission") just tries, and handles the failure if it comes. Python culture prefers EAFP — it is one step instead of two, and nothing can change between the check and the use.

```
# LBYL – look before you leap
if "email" in user:
    email = user["email"]

# EAFP – easier to ask forgiveness than permission (the Pythonic default)
try:
    email = user["email"]
except KeyError:
    email = "n/a"

# for this exact case: user.get("email", "n/a")
```

Context managers — guaranteed cleanup

Many resources need a matching "close" for every "open": files, database connections, locks. A **context manager** — anything used with the `with` statement — guarantees the cleanup happens, *no matter how the block ends*: normally, via `return`, or by raising an error. Whenever you write "open ... do ... close", reach for `with`.

```
with open("data.txt", encoding="utf-8") as f:    # closed automatically,
    text = f.read()                            # even if the block raises

from contextlib import contextmanager
import time

@contextmanager
def timer():
    start = time.perf_counter()
    try:
        yield                                # the with-block runs here
    finally:                                # ALWAYS runs
        print(f"took {time.perf_counter() - start:.3f}s")

with timer():
    sum(range(10_000_000))
```

A class can be a context manager by implementing `__enter__` and `__exit__`.
`contextlib.suppress(FileNotFoundError)` swallows one specific expected error.

12. Working with JSON

JSON (JavaScript Object Notation) is the text format the internet speaks: nearly every API response, config file, and dataset you will meet is JSON. It looks like Python dicts and lists written down as text — objects in braces, arrays in brackets, strings in double quotes. Since it is only text, the work is always converting: a real Python object → a JSON string (**serialising**) to send or save, and JSON text → Python objects (**deserialising**) to actually use. The `json` module does both.

The four functions

```
import json

json.dumps(data)           # Python object  -> JSON string
json.loads(text)           # JSON string    -> Python object
json.dump(data, f)         # Python object  -> open FILE
json.load(f)               # open FILE      -> Python object
# memory hook: the trailing s = "string"
```

Useful dumps options

```
json.dumps(data, indent=2)      # pretty-print
json.dumps(data, sort_keys=True) # stable ordering (great for diffs)
json.dumps(data, ensure_ascii=False) # keep ₹ / हिंदी readable, not \u escapes
```

Type mapping

Python	JSON	Note
<code>dict</code>	object	keys are forced to strings
<code>list</code> / <code>tuple</code>	array	tuples come BACK as lists
<code>str</code>	string	
<code>int</code> / <code>float</code>	number	
<code>True</code> / <code>False</code> / <code>None</code>	true / false / null	

Things dumps refuses — and the fix

```
json.dumps(datetime.now())      # TypeError: not JSON serializable
json.dumps({1, 2, 3})           # same - sets, Decimal, custom classes too

def json_default(obj):          # teach dumps what to do
    if isinstance(obj, datetime):
        return obj.isoformat()
    if isinstance(obj, set):
        return sorted(obj)
    raise TypeError(f"not serializable: {type(obj).__name__}")

json.dumps(payload, default=json_default)
```

Dataclasses round-trip

```
from dataclasses import dataclass, asdict

@dataclass
class User:
    name: str
    age: int

text = json.dumps(asdict(User("Priya", 22))) # out
user = User(**json.loads(text))             # back in
```

Round-trip warning: JSON knows fewer types than Python. Tuples become lists, int dict-keys become strings, datetimes need converting on both sides. Never assume `loads(dumps(x)) == x`.

13. Files & Paths

Variables disappear when a program ends. To remember anything between runs — settings, logs, saved data — a program reads and writes **files**. A **path** is a file's address on disk (`reports/2026/sales.json`). Python's `pathlib` module treats paths as smart objects instead of fragile strings: they join with `/`, know their own name and folder, and behave the same on Windows, macOS, and Linux.

`pathlib` — the modern API

```
from pathlib import Path

p = Path("data") / "users.json"    # / joins path parts on any OS
p.name        # 'users.json'
p.stem        # 'users'
p.suffix      # '.json'
p.parent      # Path('data')
p.exists(), p.is_file(), p.is_dir()
p.stat().st_size      # bytes
```

Anchor paths to the FILE, not the shell

```
HERE = Path(__file__).resolve().parent
DATA = HERE / "data" / "users.json"
# A script must find its files no matter WHERE you run it from.
# Path("data/users.json") silently depends on the current working directory.
```

Reading & writing text

```
text = DATA.read_text(encoding="utf-8")    # small files: one call
DATA.write_text("hello\n", encoding="utf-8") # replaces the whole file

with open(DATA, encoding="utf-8") as f:      # streaming, line by line
    for line in f:
        process(line.rstrip("\n"))

with open(log, "a", encoding="utf-8") as f:  # "a" = append ("w" overwrites!)
    f.write("new entry\n")
```

Always pass `encoding="utf-8"` explicitly — the default differs per OS, and that difference becomes a production bug.

JSON files — the standard patterns

```
import json

def load_json(path):
    try:
        with open(path, encoding="utf-8") as f:
            return json.load(f)
    except FileNotFoundError:
        return {} # sensible default
    except json.JSONDecodeError as e:
        raise ValueError(f"corrupt JSON in {path}: {e}") from e

def save_json(path, data): # atomic write: never half-written files
    tmp = path.with_suffix(".tmp")
    tmp.write_text(json.dumps(data, indent=2), encoding="utf-8")
    tmp.replace(path) # atomic on the same filesystem
```

Directories

```
out = Path("output")
out.mkdir(parents=True, exist_ok=True) # mkdir -p

list(HERE.glob("*.py")) # files in this dir
list(HERE.rglob("*.py")) # recursive
for p in HERE.rglob("*.json"):
    print(p, p.stat().st_size)

import shutil
shutil.copy2(src, dst) # copy with metadata
shutil.move(src, dst)
p.unlink(missing_ok=True) # delete a file
shutil.rmtree(folder) # delete a whole tree – BE CAREFUL
```

JSON Lines (.jsonl)

```
# one JSON object per line – the log/dataset format you'll meet in AI work
with open("events.jsonl", encoding="utf-8") as f:
    events = [json.loads(line) for line in f if line.strip()]
```

14. Modules & Imports

Real projects never fit in one file. Code is split into **modules** (a module is just a `.py` file) and grouped into **packages** (a folder of modules), and `import` is how one file uses another's code. Most of the confusing import errors you will ever see come from one thing: not knowing *where* Python searches for a module. That search order is explained below — read it twice.

The import forms

```
import json                # use as json.loads(...)
import numpy as np         # alias
from pathlib import Path    # pull one name in
from collections import Counter, deque
# Avoid `from x import *` – it hides where names come from.
```

What is a module? A package?

- A **module** is simply a `.py` file. `utils.py` → `import utils`.
- A **package** is a folder containing `__init__.py`. The `__init__.py` runs when the package is imported and usually re-exports the public names:

```
mypkg/
  __init__.py    # from .mymath import add    ← re-export
  mymath.py      # def add(a, b): ...

# consumer:
from mypkg import add
```

How Python FINDS modules — `sys.path`

Imports are resolved by searching, in order: (1) **the directory of the script you ran**, (2) any `PYTHONPATH` entries, (3) the standard library, (4) installed packages (`site-packages` — your venv).

The classic trap: because your script's own folder is searched *first*, a local file named like a stdlib module — `json.py`, `random.py`, `time.py` — **shadows the real one** and breaks every import of it with confusing errors. Never name your files after stdlib modules.

Modules run once and are cached

```
# config.py – top-level code
print("loading config")    # prints ONE time only, on first import
settings = {"debug": True}

# Every importer gets the SAME module object (a singleton) –
# mutate config.settings in one file and every other file sees it.
```


15. Type Hints

Python never forces you to declare types — but in team code, knowing what a function accepts and returns is gold. **Type hints** let you write that down right in the signature: `def total(prices: list[float]) -> float`. Python itself ignores them while running; their value is that your editor can autocomplete and warn you, and a checker tool (`mypy`) can find type bugs *before* the code ever runs. Think of them as enforced documentation.

Hints are documentation + tooling, not enforcement

```
def total(prices: list[float], tax: float = 0.18) -> float:
    return sum(prices) * (1 + tax)

total(["a", "b"])    # runs anyway! Python does NOT check hints at runtime.
                    # A type CHECKER (mypy) catches it before you run.
```

The core vocabulary (Python 3.9 syntax)

```
from typing import Optional, Union, Callable, Literal, Final

age: int = 22
scores: list[int] = [90, 82]           # built-in generics work on 3.9
user: dict[str, str] = {"name": "P"}
point: tuple[int, int] = (3, 5)

email: Optional[str] = None            # str OR None (3.9: NOT `str | None`)
raw: Union[int, str] = "42"            # one of several types
Mode = Literal["dev", "prod"]          # only these exact strings
MAX_RETRIES: Final = 3                 # a constant – checker forbids reassignment
on_done: Callable[[int, str], bool]    # function taking (int, str) -> bool
```

Version note: the `str | None` union syntax needs Python 3.10+. On 3.9 use `Optional[str]` / `Union[...]`, or add `from __future__ import annotations` at the top of the file.

TypedDict — typing JSON-shaped dicts

```
from typing import TypedDict

class UserDict(TypedDict):
    name: str
    age: int
    email: Optional[str]

def signup(u: UserDict) -> str:
    return u["name"]           # the checker knows the keys AND their types
```

Narrowing & generics (brief)

```
def describe(x: Union[int, str]) -> str:
    if isinstance(x, int):          # inside this branch the checker knows x: int
        return f"number {x + 1}"
    return x.upper()               # and here x: str

from typing import TypeVar
T = TypeVar("T")
def first(items: list[T]) -> T:    # generic: returns the SAME type it was given
    return items[0]
```

Running a checker

```
pip install mypy
mypy myfile.py          # reports type errors without running the code
```

16. Async Python (asyncio)

Async is about **waiting efficiently**. When code calls a network API, it mostly sits idle waiting for the reply. Normal code waits for one reply before starting the next request; async code starts many requests and handles each reply as it arrives — like a good waiter serving five tables instead of standing at one until the food is ready. The `async / await` keywords mark the places where code is allowed to pause and let something else run. This matters here because AI applications spend most of their time waiting on model APIs.

Coroutines don't run until awaited

```
import asyncio

async def fetch(name, seconds):
    await asyncio.sleep(seconds)    # non-blocking pause (NEVER time.sleep here)
    return f"{name} done"

fetch("a", 1)                    # does nothing — just creates a coroutine object!
asyncio.run(main())              # the one entry point that starts the event loop
```

Sequential vs concurrent

```
async def sequential():
    a = await fetch("a", 1)
    b = await fetch("b", 1)        # total: ~2 s

async def concurrent():
    a, b = await asyncio.gather(   # both run at once — total: ~1 s
        fetch("a", 1),
        fetch("b", 1),
    )
```

`gather` preserves the order of results and, by default, fails fast if any task raises (`return_exceptions=True` collects errors instead).

Tasks — start now, await later

```
async def main():
    task = asyncio.create_task(fetch("bg", 2))    # scheduled immediately
    await do_other_work()
    result = await task                          # collect it when needed
```

The traps

- **Forgetting `await`** — you get a coroutine object, not a result, and the code never runs (Python prints a "never awaited" warning).
- **Blocking calls inside async code** — `time.sleep()`, `requests.get()` freeze the entire event loop. Use `asyncio.sleep()`, an async HTTP client, or push blocking work to a thread: `await asyncio.to_thread(blocking_fn)`.
- `asyncio.run()` once per program, at the top — not inside a running loop.

- `asyncio.Semaphore(n)` limits how many coroutines run a section at once (e.g. max 5 concurrent API calls).

Why it matters here: AI applications spend most of their time waiting for LLM/API responses over the network — exactly the workload `asyncio` is built for. You will meet `async/await` everywhere in GenAI code.

17. Standard Library Tour

Python's slogan is "batteries included": a huge **standard library** ships with the language itself — counting, dates, regular expressions, hashing, CSV, logging and much more, with nothing to install. Before reaching for `pip install`, check whether the stdlib already does the job. These are the modules you will actually use week to week.

collections

```
from collections import Counter, defaultdict, deque, namedtuple

Counter(words).most_common(3)          # top-3 frequency
groups = defaultdict(list)             # auto-create missing values
q = deque([1, 2, 3])                   # real queue: O(1) at both ends
q.popleft()                             # list.pop(0) is O(n) – use deque for queues
Point = namedtuple("Point", "x y")     # lightweight record: Point(3, 5).x
```

datetime

```
from datetime import datetime, timedelta, timezone

now = datetime.now(timezone.utc)        # timezone-AWARE now
now.isoformat()                         # '2026-09-11T07:30:00+00:00' – use ISO everywhere
datetime.fromisoformat("2026-09-11T13:00:00")
now.strftime("%d %b %Y, %H:%M")         # format for humans
datetime.strptime("11/09/2026", "%d/%m/%Y") # parse a known format

deadline = now + timedelta(days=7)      # date arithmetic
(deadline - now).total_seconds()

from zoneinfo import ZoneInfo           # real timezones (3.9+)
now.astimezone(ZoneInfo("Asia/Kolkata"))

import time
start = time.perf_counter()             # measuring elapsed time
elapsed = time.perf_counter() - start
```

re — regular expressions

```
import re

m = re.search(r"(\w+)@(\w+\.\w+)", text)    # first match anywhere (or None)
if m:
    user, domain = m.group(1), m.group(2)

m = re.search(r"(?P<user>\w+)@(?P<host>\S+)", text)    # named groups
m.group("user")

re.findall(r"#(\w+)", tweet)                # every match as a list
re.sub(r"\s+", " ", messy)                  # replace
re.fullmatch(r"\d{10}", phone)              # must match the WHOLE string

PATTERN = re.compile(r"\d+")                # compile once if reused in a loop
```

`search` finds a match anywhere; `match` only at the start; `fullmatch` must cover the whole string. Use raw strings (`r"..."`) for every pattern. And don't reach for `regex` when `in` , `startswith` , or `split` already does the job.

Grab-bag you will actually use

```
import random
random.randint(1, 6); random.choice(items); random.shuffle(deck)
random.seed(42)                                # reproducible randomness

import math, statistics
math.ceil(4.2); math.floor(4.8); math.sqrt(16)
statistics.mean(nums); statistics.median(nums)

import uuid
str(uuid.uuid4())                                # unique id: 'e4f7...'

import hashlib
hashlib.sha256(b"secret").hexdigest()

import os
os.environ.get("API_KEY")                        # read environment variables (with default!)

import csv
with open("users.csv", encoding="utf-8", newline="") as f:
    for row in csv.DictReader(f): # each row is a dict keyed by the header
        print(row["name"])

from enum import Enum
class Status(Enum):
    ACTIVE = "active"
    BANNED = "banned"

from functools import lru_cache, partial
@lru_cache(maxsize=None)                        # memoise a pure function
def fib(n): ...
double = partial(multiply, 2)                  # pre-fill arguments
```

logging — stop print-debugging real code

```
import logging
logging.basicConfig(level=logging.INFO,
                    format="%(asctime)s %(levelname)s %(message)s")
log = logging.getLogger(__name__)

log.debug("detail for devs")          # hidden unless level=DEBUG
log.info("user %s signed up", name)
log.warning("retrying...")
log.error("payment failed")
# Levels can be filtered without deleting lines — print() can't do that.
```

How to keep improving: re-type the examples instead of reading them, break them on purpose and read the tracebacks, and try `help(thing)` in the REPL before searching the web. The goal for this training: be able to read any Python file in a real codebase and know what every line does.

Learn more — good places to go deeper

- **W3Schools Python** — w3schools.com/python — short pages with a "Try it yourself" editor for every topic in these notes. Best for a quick second explanation when something here felt too fast.
- **The official Python tutorial** — docs.python.org/3/tutorial — written by the Python team itself; more depth, and the library reference next to it answers "what arguments does this function take?" for every stdlib module.
- **Real Python** — realpython.com — practical, well-written articles when you want one topic (decorators, asyncio, pathlib) explained end-to-end with real examples.